

Experiment -1

Linux Programming Environment & Basic Commands

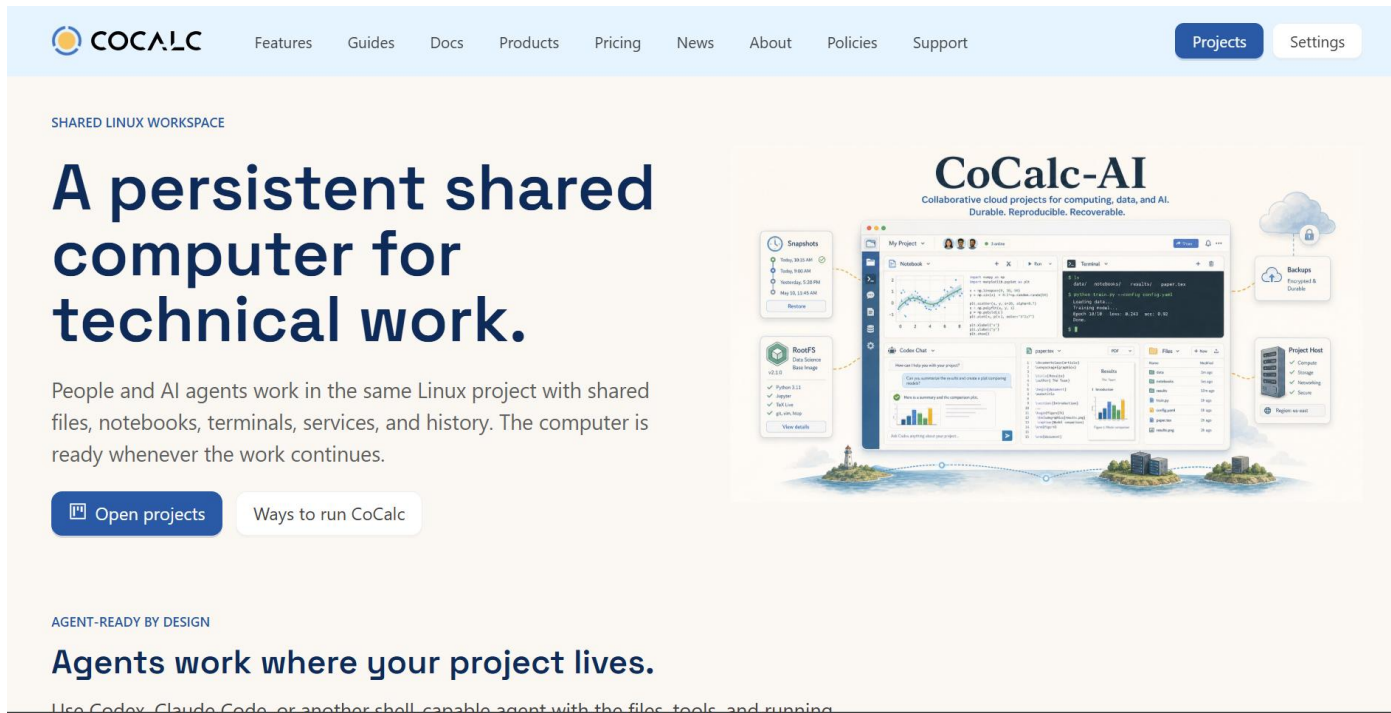
A) Setting up of Linux Programming Environment

We can do C programming online or offline according to the needs. It is required for us to keep working on a Linux Environment.

Online:

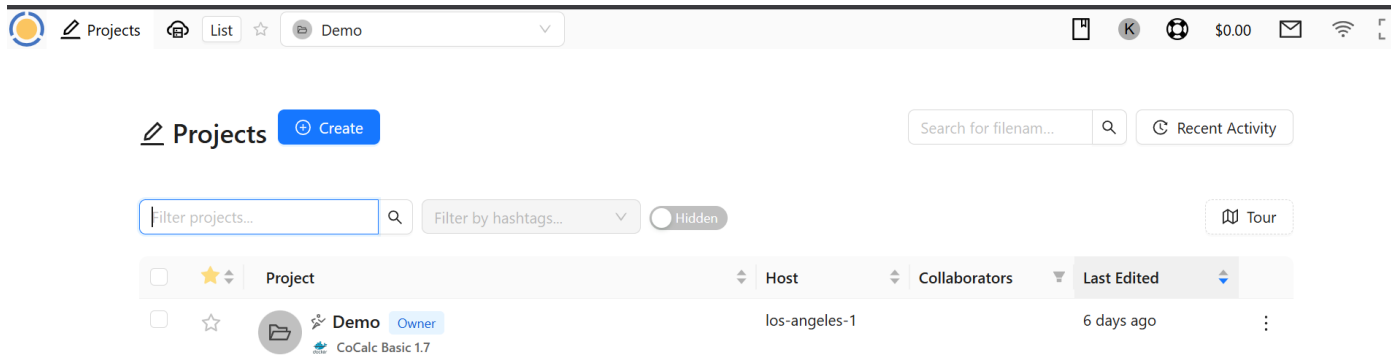
Cocalc.ai

1. Open the coclac.ai website and login using your official student mail id provided by the college



The image shows the CoCalc-AI website landing page. At the top is a navigation bar with links: COCALC, Features, Guides, Docs, Products, Pricing, News, About, Policies, Support, Projects, and Settings. The main heading is "A persistent shared computer for technical work." Below this, it says "People and AI agents work in the same Linux project with shared files, notebooks, terminals, services, and history. The computer is ready whenever the work continues." There are two buttons: "Open projects" and "Ways to run CoCalc". Below that, it says "AGENT-READY BY DESIGN" and "Agents work where your project lives." At the bottom, there is a small text: "Use Codex, Claude Code, or another shell capable agent with the files, tools, and running..."

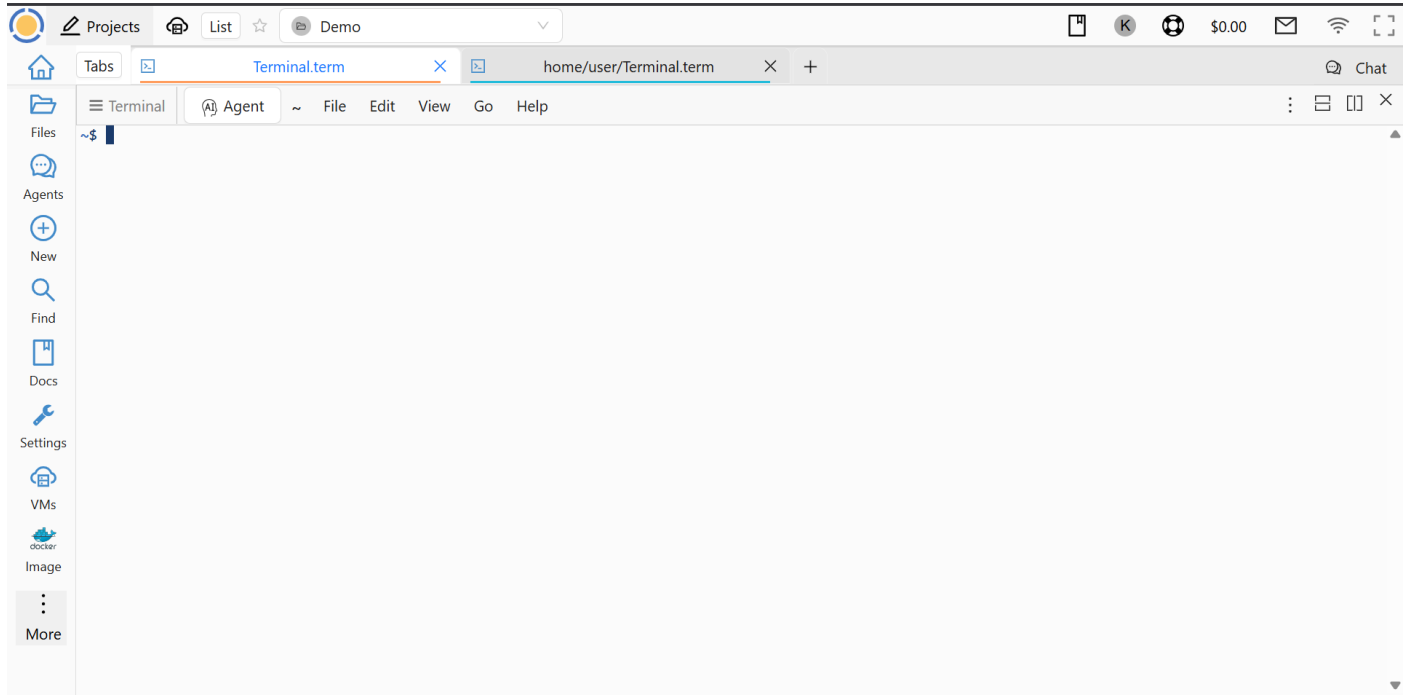
2. After completing the sign-up start with a new project.



The image shows the CoCalc Projects page. At the top, there is a navigation bar with links: Projects, List, Demo, and a dropdown menu. Below this, there is a "Create" button and a search bar. The main content area shows a list of projects. The first project is "Demo" with the owner "Owner". It is located in "los-angeles-1" and was last edited "6 days ago". The project is a "CoCalc Basic 1.7" project.

Project	Host	Collaborators	Last Edited
Demo CoCalc Basic 1.7	los-angeles-1		6 days ago

3. After some time, the interface will change into a terminal-like interface with a dollar on the screen, which confirms you are in the “virtual linux environment” on which you can work without installing linux OS on your personal machine



Offline:

Methods: Dual boot, Single boot, Bootable Pen drive, VM

1. Dual Boot: Dual-Boot method refers to a method in which two separate OS to live on the same machine without any interference between the two... so both the machine gets direct access to the hardware without any layering and latency
 2. Single Boot: As the name refers, it is opposite of the dual boot method of booting/operating the system... only one single OS uses the hardware of the computer singularly. It offers a smoother booting experience the direct executions.
 3. Bootable Pen Drive: when a flash drive contains a bootloader and an OS... then the hardware treats it as an internal drive and let's the user use the drive's OS, the drive operated the hardware without any interference from any other OS, gets the access to the entire hardware
 4. VM: It helps mimic a computer and let's the user operate a secondary OS on the primary OS... just like a layer... The second OS (on the VM) gives command to the primary OS and the primary OS runs directly on the hardware. So VM acts as a connector between the primary and the secondary OS. It has two branches which are discussed below.
1. Virtual Machine Manager (Hypervisor)

Type-1 (Bare-metal)

Bare-metal hypervisors are the ones which are directly installed on the hardware and they directly communicate with the machine instead of sitting on top of another software. The virtual machine makes direct communication and prevents latency in the system and the OS

E.g. are VMware and MS Hyper-V

Type-2 (Hosted)

These types of hypervisors are the one which sit on top of any standard OS unlike the bare-metal Hypervisor's they have to communicate with the computer hardware via the primary operating system, which can lead to latency and multiple CPU requests.

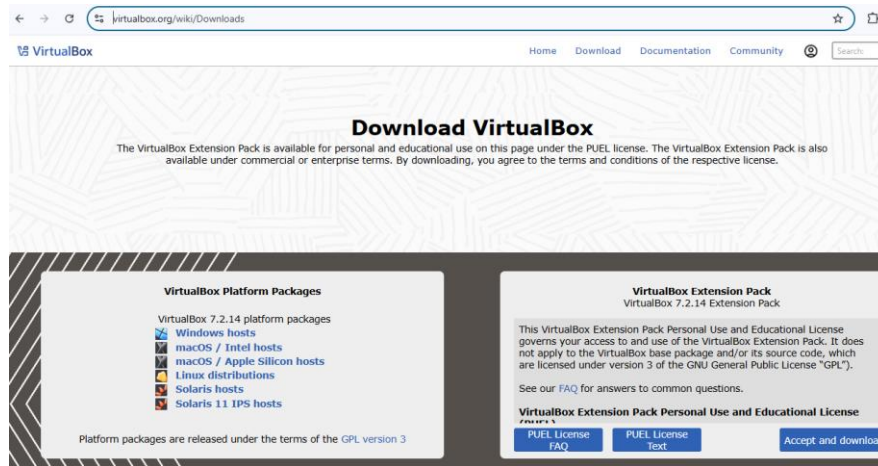
VMM -> Hypervisor -> Primary OS -> Physical hardware

E.g Oracle Virtual-box, VM Ware.

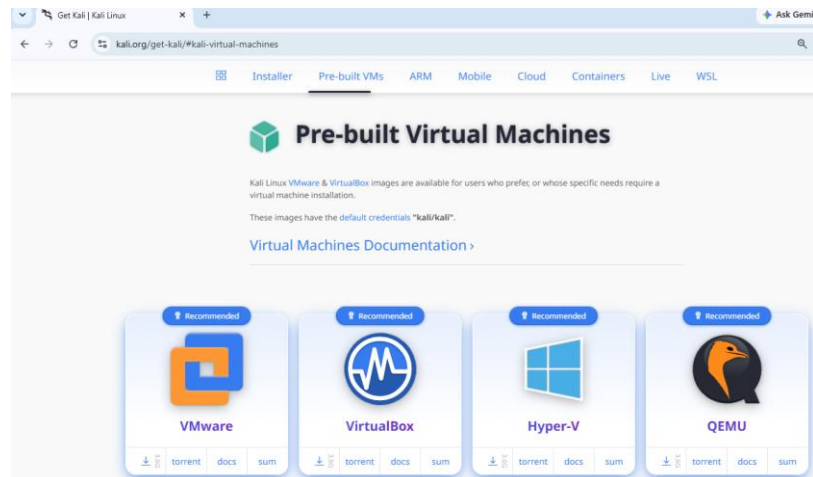
We would be using Oracle Virtual-box.

2. Installation of VMM & VM

Step-1: Download Oracle Virtual Box for “Windows hosts”

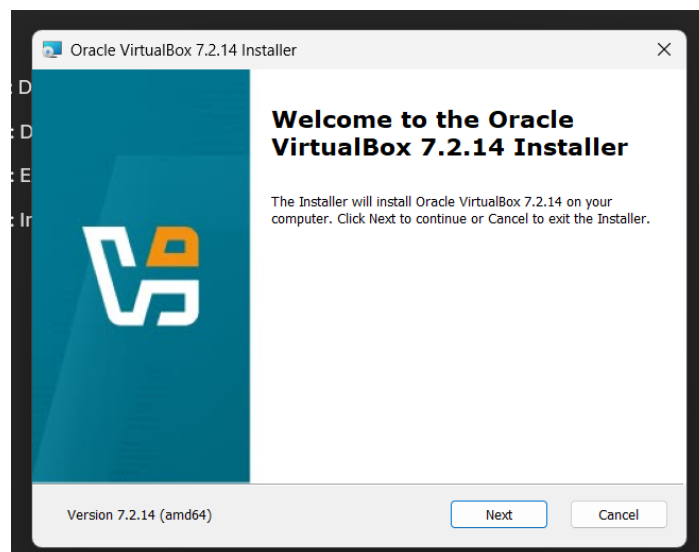


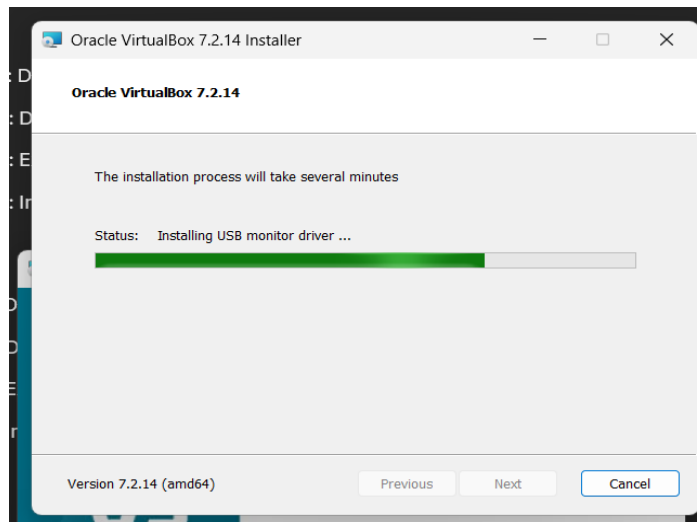
Step-2: Download Pre-built Kali Linux VM for Virtualbox



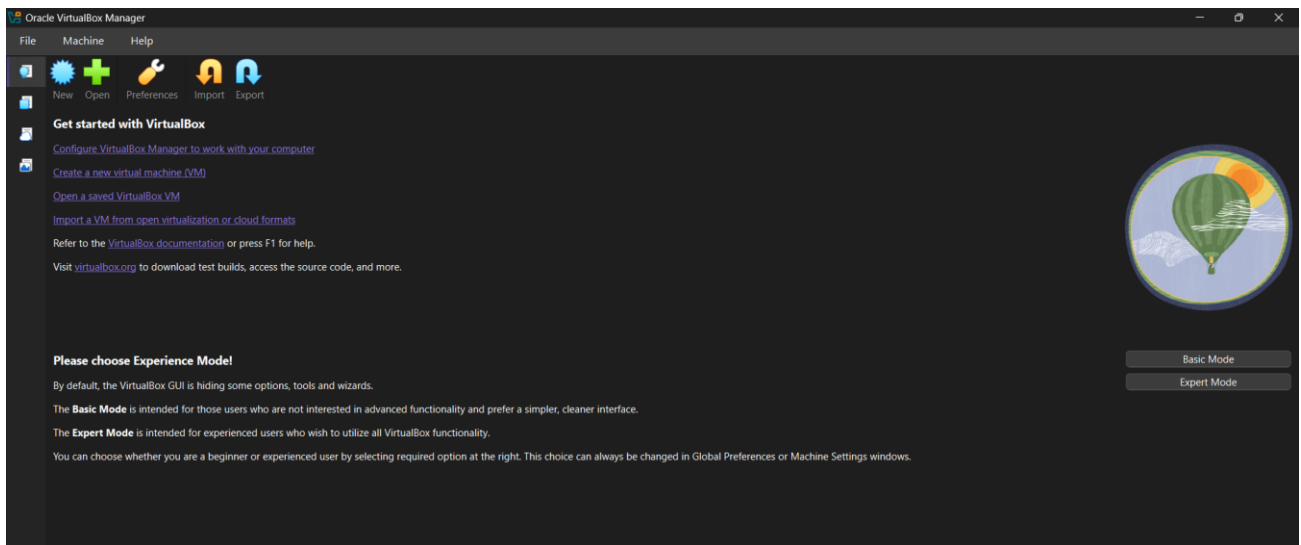
Step-3: Extracted the Kali Linux downloaded file

Step-4: Install Virtualbox

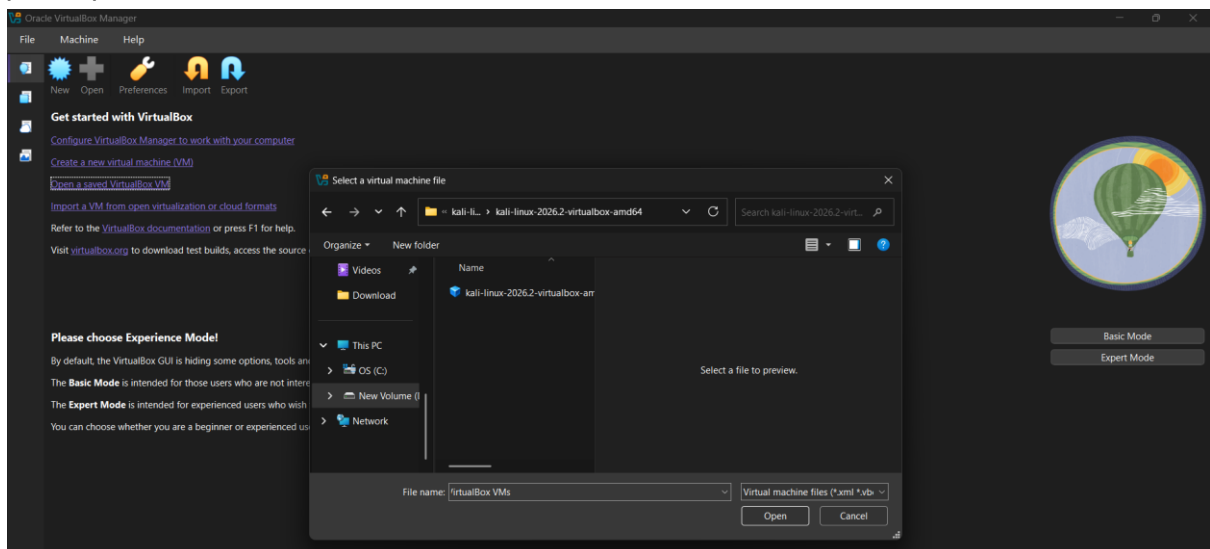


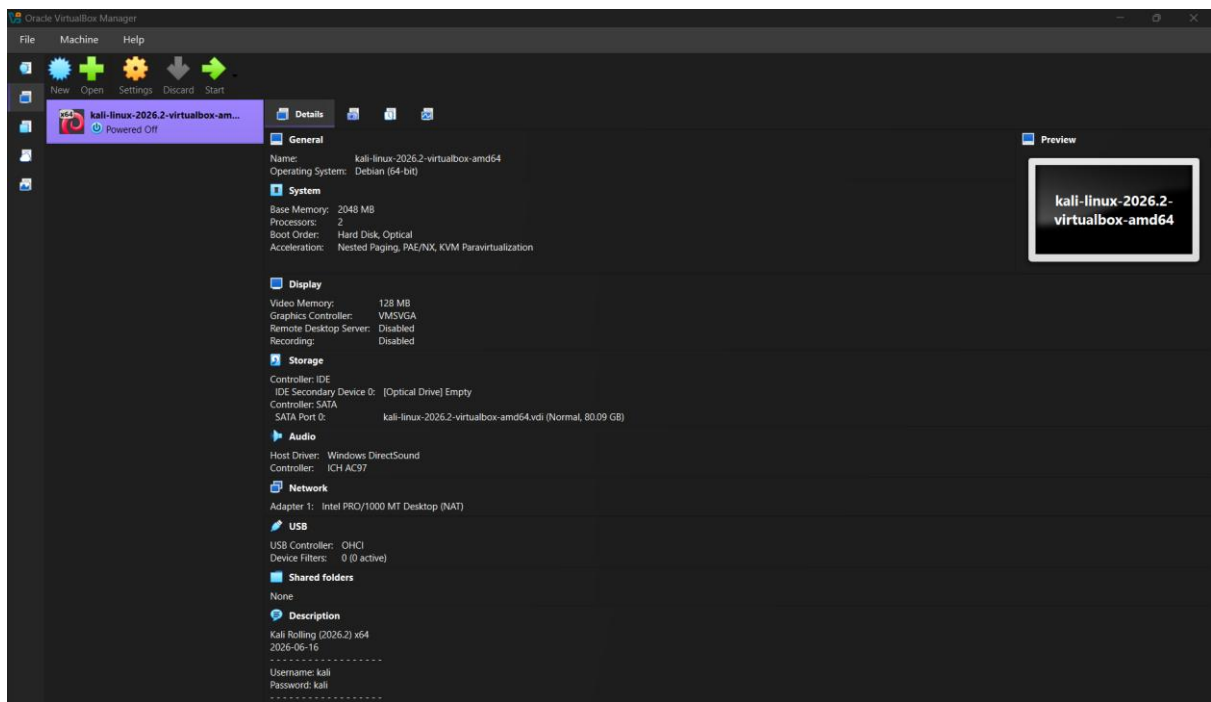


Step-5: Open Virtual box

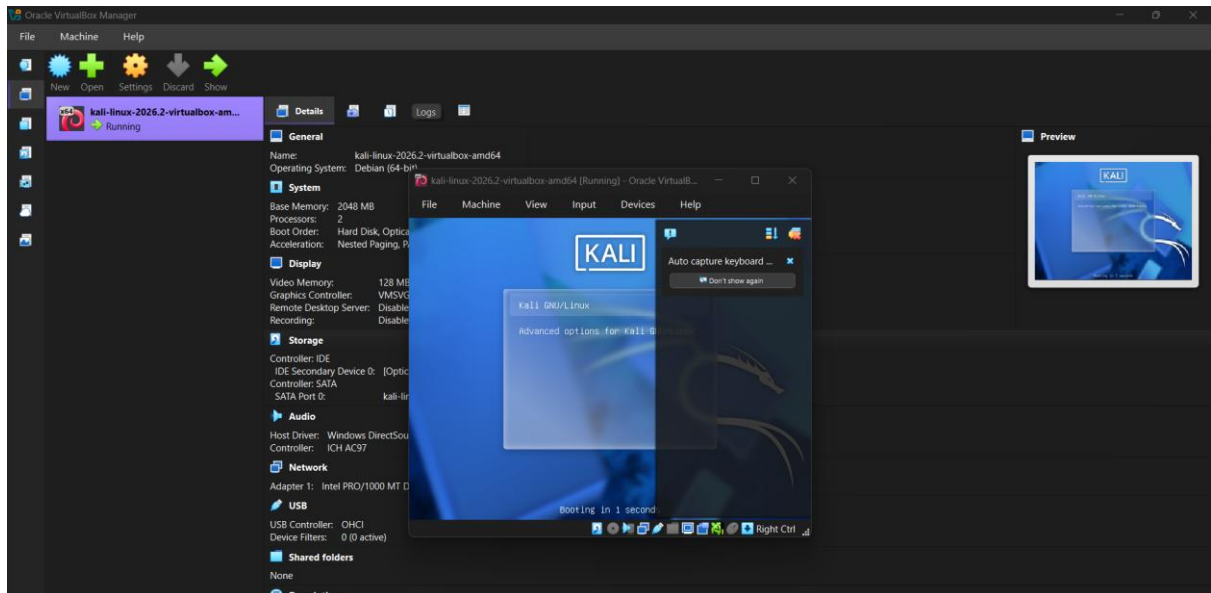


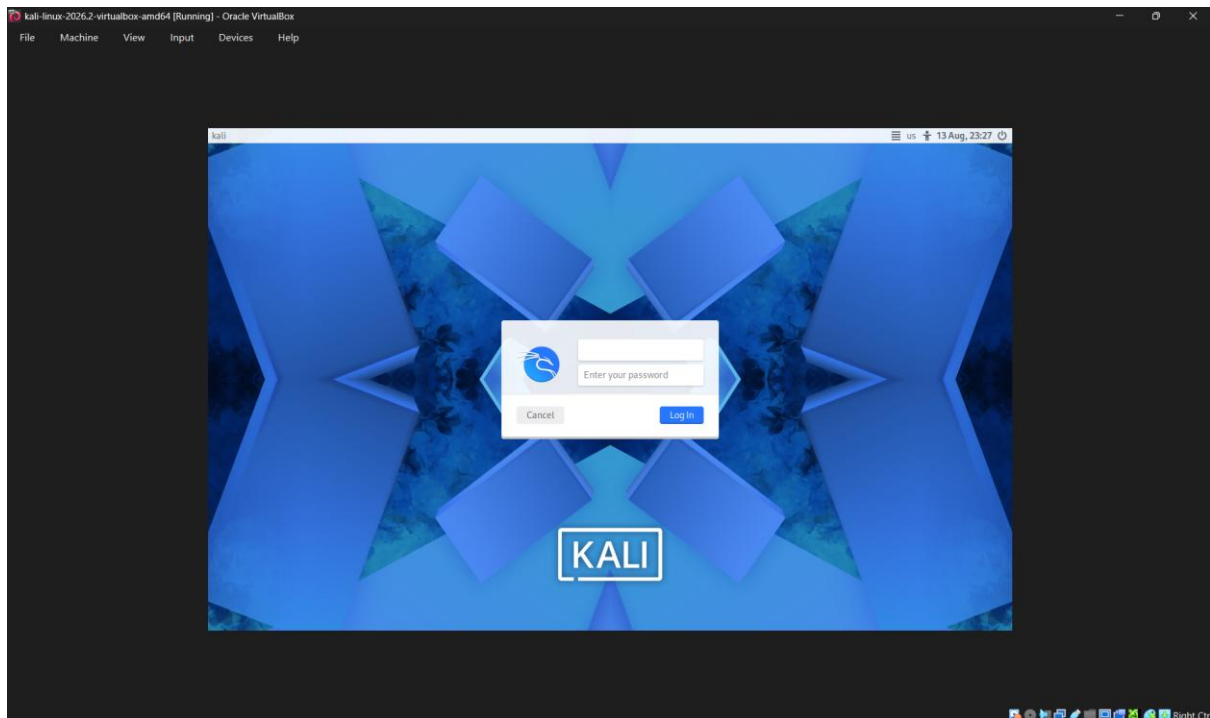
Step-6: Open the “saved VM”





Step-7: Start the VM





Thus, the installation of VMM (Hypervisor Type-2 Oracle VirtualBox) & VM (Kali Linux) is completed.

Step-8: Open terminal and create the following simple program, compile and execute to understand the working of the environment.

B) Working with Linux Commands

Execute the following commands in your environment, take output snippets, and provide a brief explanation.

- Terminal basics: pwd, mkdir, rmdir, dir, ls, cd, cd .., clear, history, man, whoami
- Files: vi, file, rm, cp, mv, cat, more, less, head, tail, chmod
- Redirection: >, >>, <

Working Linux Commands

```
kali@kali: ~/liak
Session Actions Edit View Help
(kali@kali)-[~/Desktop]
$ pwd #Tells you the directory are working on
/home/kali/Desktop
(kali@kali)-[~/Desktop]
$ cd /home/kali/liak #Changes the active dir
(kali@kali)-[~/liak]
$ pwd
/home/kali/liak
(kali@kali)-[~/liak]
$ dir #displays all the files present
agg.out a.out doubt.c first.c try.c
(kali@kali)-[~/liak]
$
```

```
(kali@kali)-[~/liak/assign]
$ cd /home/kali/liak #changes the directory
(kali@kali)-[~/liak]
$ rmdir assign # removes the dir named "Assign"
(kali@kali)-[~/liak]
$ dir
agg.out a.out doubt.c first.c try.c
(kali@kali)-[~/liak]
$ cd /home/kali
(kali@kali)-[~/] # no dir names assign showing
$ dir
Desktop Downloads liak Pictures Public Videos
Documents kali Music Projects Templates
```



```
(kali@kali)-[~/liak]
$ more doubt.c #displays the entire code
#include <stdio.h>
int calpercent(int sci,int math,int sans);
int main(void)
{
    int sci,math,sans;
    printf("Enter the marks of all the subjects\n");
    scanf("%d,%d,%d",&sci,&math,&sans);
    // printf("The agg is %f\n",percentage);
    // printf("%f",calpercent(sci,math,sans));
    calpercent(sci,math,sans);
    return 0;
}
int calpercent(int sci, int math, int sans)
{
    int total=sci+math+sans;
    int maxtotal=300;
    /* printf("Your aggregate is:\n");*/
    float percentage=(1.0*total/maxtotal)*100.0;
    printf("Agg is %f\n",percentage);
    return percentage;
}

(kali@kali)-[~/liak]
$ head doubt.c #displays the first 10 line of the code
#include <stdio.h>
int calpercent(int sci,int math,int sans);
int main(void)
{
    int sci,math,sans;
    printf("Enter the marks of all the subjects\n");
    scanf("%d,%d,%d",&sci,&math,&sans);
    // printf("The agg is %f\n",percentage);
    // printf("%f",calpercent(sci,math,sans));
    calpercent(sci,math,sans);

(kali@kali)-[~/liak]
$ tail doubt.c #displays the last 10 lines of the code
int calpercent(int sci, int math, int sans)
{
    int total=sci+math+sans;
    int maxtotal=300;
    /* printf("Your aggregate is:\n");*/
    float percentage=(1.0*total/maxtotal)*100.0;
    printf("Agg is %f\n",percentage);
    return percentage;
}
```

```
(kali@kali)-[~/liak]
$ dir # shows the directories
agg.out a.out doubt.c first.c try.c

(kali@kali)-[~/liak]
$ rm doubt.c # Permanently removes the dir

(kali@kali)-[~/liak]
$ dir
agg.out a.out first.c try.c
```

```
$ file try.c #shows the format of the file
try.c: C source, ASCII text

(kali@kali)-[~/liak]
$ cat try.c # displays all the text data in the file
#include<stdio.h>
int main(void){
    printf("Hello");
}

(kali@kali)-[~/liak]
$ rm agg.out # displays all the text data in the file

(kali@kali)-[~/liak]
$ dir
a.out first.c try.c

(kali@kali)-[~/liak]
$ cp try.c copp.c #created a copy of already existing file

(kali@kali)-[~/liak]
$ dir
a.out copp.c first.c try.c

(kali@kali)-[~/liak]
$ mv try.c proj.c #helps in renaming the files

(kali@kali)-[~/liak]
$ dir
a.out copp.c first.c proj.c
```